ASSIGNMENT--3

Task 1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Answer:

I made the login form with HTML, CSS and JavaScript (file: login.html). The hardcoded credentials are username "admin" and password "Admin@123". The page shows "Access Granted" only when both fields match, and otherwise shows "Access Denied".

Code (login.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Zomato Admin Panel - Login</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f4f4; display: flex;
justify-content: center; align-items: center; height: 100vh; margin: 0; }
    .box { background: #fff; padding: 30px; border-radius: 8px; width: 300px; box-
shadow: 0 2px 8px rgba(0,0,0,0.2); }
    h2 { text-align: center; color: #e23744; margin-top: 0; }
    input { width: 100%; padding: 8px; margin: 8px 0 14px; box-sizing: border-box; }
    button { width: 100%; padding: 10px; background: #e23744; color: #fff; border:
none; border-radius: 4px; cursor: pointer; }
    #msg { text-align: center; margin-top: 14px; font-weight: bold; }
    .ok { color: green; } .bad { color: red; }
  </style>
</head>
<body>
  <div class="box">
    <h2>Zomato Admin Login</h2>
    <form id="loginForm">
      <label>Username</label>
      <input type="text" id="username" placeholder="Enter username" required>
      <label>Password</label>
      <input type="password" id="password" placeholder="Enter password" required>
      <button type="submit">Login</button>
    </form>
    <div id="msg"></div>
  </div>

  <script>
    // Hardcoded admin credentials (only for this mock demo)
    const ADMIN_USER = "admin";
    const ADMIN_PASS = "Admin@123";

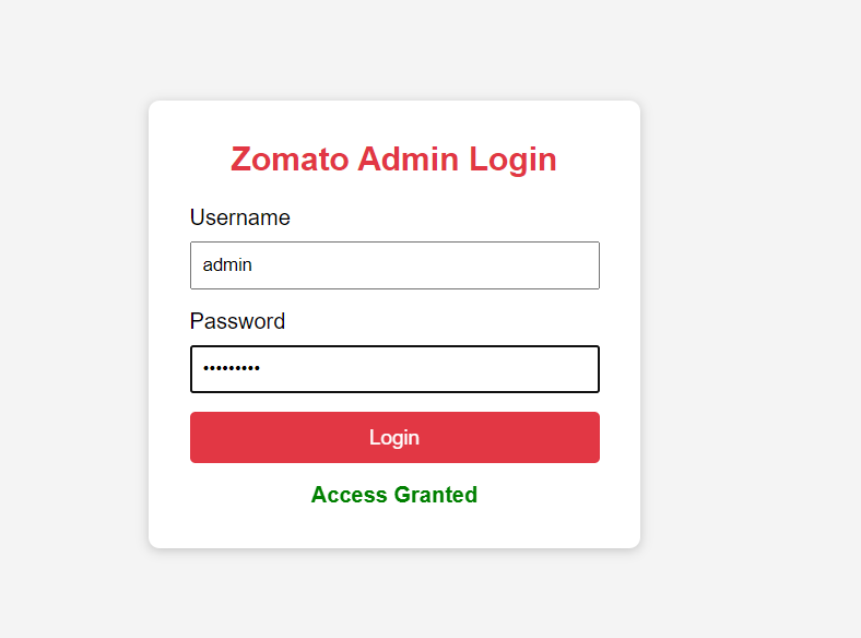
    document.getElementById("loginForm").addEventListener("submit", function (e) {
      e.preventDefault();
      const user = document.getElementById("username").value;
      const pass = document.getElementById("password").value;
      const msg = document.getElementById("msg");

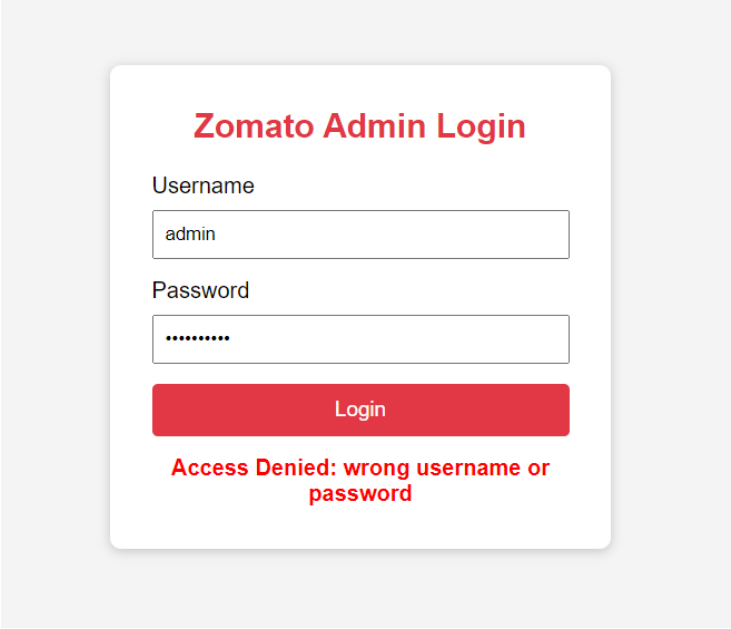
      if (user === ADMIN_USER && pass === ADMIN_PASS) {
        msg.textContent = "Access Granted";
        msg.className = "ok";
      } else {
```

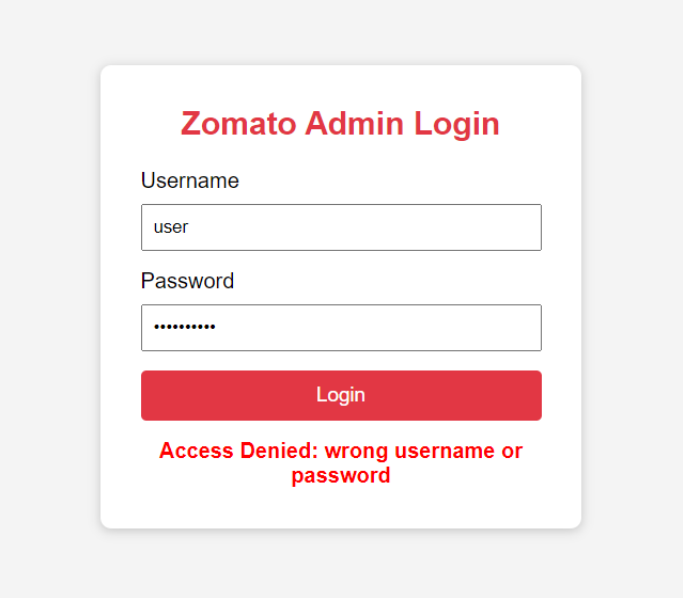
```
        msg.textContent = "Access Denied: wrong username or password";
        msg.className = "bad";
      }
    });
  </script>
</body>
</html>
```

Test results:

```
Username: admin    Password: Admin@123    ->  Access Granted
Username: admin    Password: wrong        ->  Access Denied: wrong username or
password
Username: user     Password: Admin@123    ->  Access Denied: wrong username or
password
```







Note: Hardcoding a password in front-end code is only for this mock demo. In a real application the check must be done on the server and the password must be stored as a hash.

Task 2. Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

Answer:

I used a dictionary that maps each role to a set of allowed resources. If the role is not in the dictionary, the function returns False, so unknown roles get no access by default.

Code (access.py):

```
# Role-based access control (RBAC) for a Zomato-like app

# Dictionary that maps each role to the resources it can use
ROLE_PERMISSIONS = {
    "customer": {"order_history"},
    "restaurant_owner": {"menu_edit", "order_history"},
    "admin": {"menu_edit", "order_history", "admin_dashboard"},
}


def check_access(user_role, resource):
    """Return True if user_role is allowed to access the resource, else
False."""
```

```
    allowed = ROLE_PERMISSIONS.get(user_role, set())   # unknown role -> no
access
    return resource in allowed


if __name__ == "__main__":
    tests = [
        ("customer", "menu_edit"),
        ("customer", "order_history"),
        ("customer", "admin_dashboard"),
        ("restaurant_owner", "menu_edit"),
        ("restaurant_owner", "order_history"),
        ("restaurant_owner", "admin_dashboard"),
        ("admin", "admin_dashboard"),
        ("guest", "order_history"),      # unknown role
    ]
    for role, resource in tests:
        print(f"check_access('{role}', '{resource}') -> {check_access(role,
resource)}")
```

Output:

```
check_access('customer', 'menu_edit') -> False
check_access('customer', 'order_history') -> True
check_access('customer', 'admin_dashboard') -> False
check_access('restaurant_owner', 'menu_edit') -> True
check_access('restaurant_owner', 'order_history') -> True
check_access('restaurant_owner', 'admin_dashboard') -> False
check_access('admin', 'admin_dashboard') -> True
check_access('guest', 'order_history') -> False
```

Explanation: A customer can only see order history. A restaurant owner can edit the menu and see order history, but cannot open the admin dashboard. Only the admin can open the admin dashboard.

Task 3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Answer:

The program first asks for the password (step 1). If it is correct, it generates a random 6-digit OTP, prints it in the console (instead of sending an SMS) and asks the user to type it (step 2). "Login Successful" is printed only if the OTP matches. The demo password is "Paytm@123". I used the secrets module because it gives a more secure random number than the random module.

Code (otp_login.py):

```
CORRECT_PASSWORD = "Paytm@123"      # hardcoded only for this demo
def generate_otp():
    """Generate a random 6-digit OTP."""
    return str(secrets.randbelow(900000) + 100000)
```

```
def login():
    # Step 1: password
    password = input("Enter password: ")
    if password != CORRECT_PASSWORD:
        print("Wrong password. Login failed.")
        return False

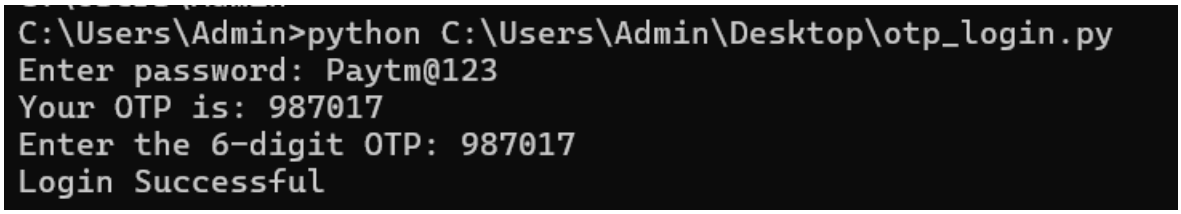
    # Step 2: generate and show OTP in the console (instead of sending an
SMS)
    otp = generate_otp()
    print(f"Your OTP is: {otp}")

    # Step 3: ask the user to enter the OTP
    entered = input("Enter the 6-digit OTP: ")
    if entered == otp:
        print("Login Successful")
        return True
    print("Invalid OTP. Login failed.")
    return False

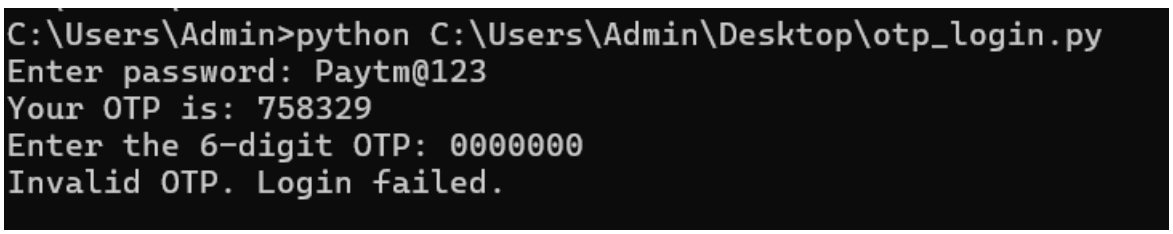
if __name__ == "__main__":
    login()
```

Output (three test runs):

```
--- Run 1: correct password, correct OTP ---
Enter password: Paytm@123
Your OTP is: 987017
Enter the 6-digit OTP: 987017
Login Successful
```




```
--- Run 2: correct password, wrong OTP ---
Enter password: Paytm@123
Your OTP is: 758329
Enter the 6-digit OTP: 000000
Invalid OTP. Login failed.
```




```
--- Run 3: wrong password ---
Enter password: abc123
```

```
Wrong password. Login failed.
```

```
C:\Users\Admin>python C:\Users\Admin\Desktop\otp_login.py
Enter password: abc123
Wrong password. Login failed.
```

Task 4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

Answer:

I wrote the function in JavaScript using the filter() method. It converts lastLogin to a date, finds how many days ago it was from 2024-06-10, and keeps the user only if isPremium is true and the difference is between 0 and 7 days (7 days ago is included).

Code (premium.js):

```
// Return premium users who logged in within the last 7 days
const TODAY = "2024-06-10";            // given in the question

function getActivePremiumUsers(users) {
  const today = new Date(TODAY + "T00:00:00Z");
  const msPerDay = 24 * 60 * 60 * 1000;

  return users.filter((user) => {
    const last = new Date(user.lastLogin + "T00:00:00Z");
    const daysAgo = (today - last) / msPerDay;
    // premium AND logged in 0 to 7 days ago (future dates are ignored)
    return user.isPremium && daysAgo >= 0 && daysAgo <= 7;
  });
}

// Sample data
const users = [
  { username: "aman",   isPremium: true,  lastLogin: "2024-06-09" },   // 1
day ago
  { username: "riya",   isPremium: false, lastLogin: "2024-06-08" },   //
not premium
  { username: "karan",  isPremium: true,  lastLogin: "2024-06-03" },   //
exactly 7 days ago
  { username: "neha",   isPremium: true,  lastLogin: "2024-05-30" },   // 11
days ago
  { username: "vikas",  isPremium: true,  lastLogin: "2024-06-10" },   //
today
  { username: "sneha",  isPremium: false, lastLogin: "2024-04-15" },   //
not premium, old
];

const result = getActivePremiumUsers(users);
```

```
console.log(result);
console.log("Usernames:", result.map((u) => u.username));
```

Output:

```
[
  { username: 'aman', isPremium: true, lastLogin: '2024-06-09' },
  { username: 'karan', isPremium: true, lastLogin: '2024-06-03' },
  { username: 'vikas', isPremium: true, lastLogin: '2024-06-10' }
]
Usernames: [ 'aman', 'karan', 'vikas' ]
```

Explanation: aman (1 day ago), karan (exactly 7 days ago) and vikas (today) are premium and recent, so they are returned. riya and sneha are not premium, and neha logged in 11 days ago, so they are removed.

Task 5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Answer:

5 Common Access Control Mistakes in Real-World Applications

No.	Mistake	Example	One-line Fix / Prevention
1	Broken Object-Level Access (IDOR)	On a Flipkart-like website, changing `order_id=1001` to `1002` reveals another customer's order details.	Always verify on the server that the logged-in user owns the requested record.
2	Checking Access Only on the Client Side	The admin button is hidden in the UI, but anyone can still access the admin API directly.	Enforce all permissions on the server, not just in the frontend.
3	Missing Role Checks on Some Endpoints	An Instagram-like app protects the admin page but forgets to protect the delete-user API.	Use centralized role-check middleware and deny access by default. top10.owasp+1
4	Trusting User-Supplied Roles or Fields	A signup request containing `"isAdmin": true` is mistakenly accepted.	Never trust roles or permissions provided by users; assign them on the server.
5	Weak Session and Token Handling	Login tokens remain valid after logout or a password change.	Use secure, short-lived tokens and invalidate sessions when required.

Rule	Meaning
Principle of Least Privilege	Give users only the permissions they need—no more.
Access Control Testing	Test with two different accounts to ensure one user cannot access another user’s private data.